UNIT-5

1. Electronic Mail

Electronic Mail (E-mail) is an application-layer service used to **send, receive, store, and manage messages electronically** through a computer network or the Internet.

Instead of sending a physical letter, a user can send a digital message from one computer or mobile device to another.

Main components

- **Sender** – person who sends the email.
- **User Agent (UA)** – software used to compose and read email.
- **Mail Server** – stores and forwards emails.
- **SMTP** – transfers email between mail servers.
- **POP3/IMAP** – allows users to retrieve emails.

Examples

- Gmail
- Outlook
- Yahoo Mail

Basic working

Suppose **Rajesh** wants to send an email to **Karthik**:

```
Rajesh
  ↓
Email Application
  ↓
Rajesh's Mail Server
  ↓
Internet
  ↓
Karthik's Mail Server
  ↓
Email Application
  ↓
Karthik
```

The message travels through several systems before reaching Karthik.

2. Architecture and Services

E-mail architecture explains **how an email travels from the sender to the receiver**.

It consists of different components that work together to send, transfer, store, and retrieve emails.

Simple flow

```
Sender
  ↓
User Agent
  ↓
Sender Mail Server
  ↓
SMTP
  ↓
Receiver Mail Server
  ↓
Mailbox
  ↓
IMAP / POP3
  ↓
Receiver User Agent
  ↓
Receiver
```

Main Components of E-Mail Architecture

An email system mainly contains:

1. **User Agent (UA)**
2. **Mail Server**
3. **Mail Transfer Agent**
4. **Mail Access Agent**
5. **Protocols such as SMTP, POP3 and IMAP**

User Agent (UA)

The **User Agent** is the software through which the user interacts with email.

Examples

- Gmail
- Microsoft Outlook
- Thunderbird
- Apple Mail

Functions

The User Agent allows users to:

- Compose messages
- Send messages
- Read messages
- Reply
- Forward

- Delete
- Save messages
- Attach files

Mail Server

A **mail server** is a computer/server that handles email messages.

It performs important functions such as:

- Receiving email
- Storing email
- Sending email
- Forwarding email
- Delivering email to the receiver

Every email account is normally associated with a mail server.

Mail Transfer Agent (MTA)

The **Mail Transfer Agent** transfers an email from **one mail server to another mail server**.

```
Sender Mail Server
        ↓
       MTA
        ↓
Receiver Mail Server
```

Mail Access Agent (MAA)

The **Mail Access Agent** allows the user to **access emails stored on the mail server**.

```
Mail Server
     ↓
  IMAP / POP3
     ↓
User's Email App
```

E-Mail Protocols

Different protocols are used for different email operations.

SMTP

Simple Mail Transfer Protocol

Used to:

- Send email from User Agent to mail server
- Transfer email between mail servers

```
Sender Server ——— SMTP ———→ Receiver Server
```

POP3

Post Office Protocol Version 3

Used by the receiver to retrieve emails from the mail server.

```
Mail Server
     ↓
    POP3
     ↓
    User
```

POP3 traditionally downloads messages to the user's device.

IMAP

IMAP keeps your emails on the mail server and lets you access and manage them from different devices.

If you read or delete an email on your mobile, the change is also reflected on your laptop.

3. User Agent

A **User Agent (UA)** is a software application that allows a user to **compose, send, receive, read, reply to, and manage e-mail messages**.

Examples

- Gmail
- Microsoft Outlook
- Thunderbird
- Apple Mail

Functions of User Agent

1. Compose

The User Agent allows the user to **create a new email**.

2. Send

After composing the message, the user can click **Send**.

The User Agent sends the message to the sender's mail server.

3. Receive

The User Agent allows users to **receive and access incoming emails** from the mail server.

Protocols such as **IMAP and POP3** are used for mail access.

4. Read

The User Agent displays the received email so that the user can read it.

5. Reply

The user can reply directly to the sender.

6. Forward

The User Agent allows a received email to be sent to another person.

7. File Attachment

Users can attach files to an email.

Examples:

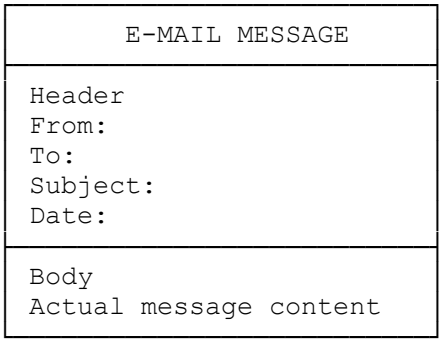
- PDF
- Word document
- Image
- Excel file
- PowerPoint presentation

4. Message Format

An **e-mail message format** defines how an email is **structured and organized** so that it can be properly sent and understood by the receiver.

An email mainly consists of **two parts**:

1. **Header**
2. **Body**



Header

The **header** contains information about the email.

Important header fields:

Field	Meaning
From	Email address of the sender
To	Email address of the receiver
Subject	Topic or purpose of the email
Date	Date and time when the email was sent
CC	Sends a copy to other recipients
BCC	Sends a hidden copy to other recipients

Example

```
From: vijay@gmail.com
To: kumar@gmail.com
Subject: Meeting
Date: 06-10-2026
```

Body

The **body** contains the **actual message or information** that the sender wants to communicate.

Example

```
Dear Kumar,

The meeting will be held at 10 AM tomorrow.

Thank you.
```

Attachments

An email can also contain **attachments**, such as:

- PDF
- Image
- Word document
- Excel file
- PowerPoint file

MIME (Multipurpose Internet Mail Extensions) is used to support different types of content and attachments in email.

Example

```
From: vijay@gmail.com
To: kumar@gmail.com
Subject: Assignment
Date: 06-10-2026

Dear Kumar,

Please find the assignment attached.

Thank you.

Attachment: Assignment.pdf
```

5. Message Transfer

Message Transfer is the process of **moving an e-mail from the sender's mail server to the receiver's mail server**.

The main protocol used for message transfer is **SMTP (Simple Mail Transfer Protocol)**.

Working of Message Transfer

Suppose **Vijay sends an email to Kumar**.

```
Vijay
  ↓
User Agent
  ↓
Vijay's Mail Server
  ↓
 SMTP
  ↓
Kumar's Mail Server
  ↓
Kumar's Mailbox
```

Step 1: Compose the message

Vijay writes an email using his **User Agent**.

Step 2: Send the message

When Vijay clicks **Send**, the message is sent to Vijay's mail server.

Step 3: Find receiver's mail server

The sender's mail server determines the mail server responsible for Kumar's email address.

Step 4: Transfer using SMTP

The sender's mail server transfers the message to Kumar's mail server using **SMTP**.

Step 5: Store the message

Kumar's mail server stores the message in his **mailbox**.

Protocols Used

SMTP

SMTP is an application-layer protocol used to **send and transfer email messages**.

It provides reliable message transfer using **TCP**.

6. Final Delivery

Final Delivery is the process of **delivering an email from the receiver's mail server to the receiver's email application (User Agent)**.

After the email reaches the receiver's mail server, it is stored in the receiver's **mailbox**. The receiver then accesses the email using **POP3 or IMAP**.

Working of Final Delivery

Suppose **Vijay sends an email to Kumar**:

```
Vijay
  ↓
Vijay's Mail Server
  ↓ SMTP
Kumar's Mail Server
  ↓
Kumar's Mailbox
  ↓
POP3 / IMAP
  ↓
Kumar's User Agent
  ↓
Kumar reads the email
```

Step 1: Email reaches receiver's server

The sender's mail server transfers the email to the receiver's mail server using **SMTP**.

Step 2: Email is stored

The receiver's mail server stores the email in the receiver's **mailbox**.

Step 3: Receiver checks email

The receiver opens an email application such as Gmail or Outlook.

Step 4: Email is accessed

The User Agent uses **POP3 or IMAP** to access the email from the mail server.

Step 5: Receiver reads the email

The email is displayed in the receiver's email application.

Protocols Used

POP3

Post Office Protocol version 3

- Retrieves emails from the mail server.
- Traditionally downloads messages to the user's device.

IMAP

Internet Message Access Protocol

- Allows users to access and manage emails while they remain on the server.
- Useful when the same email account is used on multiple devices.

7. World Wide Web (WWW)

The **World Wide Web (WWW)** is a collection of interconnected web pages and resources that are accessed through the Internet using a web browser.

Main Components of WWW

1. **Web Browser**
   Software used to access and display web pages.
   Examples: Chrome, Firefox, Edge.
2. **Web Server**
   A computer/software that stores and provides web pages and other resources to users.
3. **Web Page**
   A document displayed in a browser. It may contain text, images, videos, audio, and hyperlinks.
4. **URL (Uniform Resource Locator)**
   It is the address used to locate a web resource.
   Example: https://www.google.com
5. **HTTP/HTTPS**
   - **HTTP** is used for communication between browser and web server.
   - **HTTPS** provides secure communication using encryption.

6. **HTML**
   HyperText Markup Language is used to create and structure web pages.

Working of WWW

```
User
  ↓
Web Browser
  ↓
Enter URL
  ↓
DNS finds Web Server
  ↓
HTTP/HTTPS Request
  ↓
Web Server
  ↓
HTTP/HTTPS Response
  ↓
Web Page displayed in Browser
```

Example

When a user enters **www.google.com** in a browser:

1. Browser receives the URL.
2. DNS finds the IP address of Google's server.
3. Browser sends an HTTP/HTTPS request.
4. Web server processes the request.
5. Server sends the required web page.
6. Browser displays the page to the user.

8. World Wide Web Architectural Overview

Web Architecture describes the basic structure of the Web and explains how a **client (web browser)** communicates with a **web server** to access web resources.

Basic Architecture

```
        Client
    (Web Browser)
          |
          | HTTP / HTTPS Request
          ↓
     Web Server
          |
          | Request Processing
          ↓
   Web Application
          |
          ↓
      Database
          |
          ↓
```

```
Web Server
     |
     | HTTP / HTTPS Response
     ↓
   Client
(Web Browser)
```

Main Components

1. Client

- The client is the user's device running a web browser.
- Examples: Chrome, Firefox, Edge.
- It sends requests for web pages or other resources.

2. Web Browser

- Provides an interface for accessing websites.
- Sends HTTP/HTTPS requests and displays the received content.

3. Web Server

- Receives requests from clients.
- Stores and delivers web resources such as HTML files, images, and videos.
- Example: Apache, Nginx.

4. Web Application

- Processes requests that require dynamic operations.
- It can perform calculations, authentication, transactions, etc.

5. Database

- Stores information required by web applications.
- Example: student details, product information, user accounts.

6. HTTP/HTTPS

- HTTP is the communication protocol between the browser and server.
- HTTPS is the secure version of HTTP.

7. URL

- A URL identifies the location of a web resource.
- Example: https://www.example.com/index.html

Working of Web Architecture

1. User enters a **URL** in the browser.
2. Browser identifies the required web server.
3. Browser sends an **HTTP/HTTPS request**.

4. Web server receives and processes the request.
5. If required, the web application accesses the **database**.
6. Server prepares the response.
7. Response is sent back to the browser.
8. Browser displays the webpage to the user.

9. Static Web Pages and Dynamic Web Pages

Static Web Page

A **static web page** is a webpage whose content is **fixed and stored on the web server**. The same content is generally displayed to every user.

Working

```
User
  ↓
Web Browser
  ↓
Web Server
  ↓
Stored HTML Page
  ↓
Web Browser
  ↓
Web Page displayed
```

Features

- Content is fixed.
- Usually created using **HTML and CSS**.
- Same page is generally shown to all users.
- Easy to develop and maintain.
- Loads quickly because no database processing is normally required.
- Suitable for simple websites.

Examples

- College **About Us** page
- Contact Us page
- Company profile
- Personal portfolio
- College history page

Example:
A college "About Us" page may show the same college information to every visitor.

Dynamic Web Page

DefinitionA **dynamic web page** is a webpage whose content is **generated or changed according to the user, request, data, or other conditions**.

Working

```
User
  ↓
Web Browser
  ↓
Web Server
  ↓
Web Application
  ↓
Database
  ↓
Web Application
  ↓
Web Server
  ↓
Web Browser
```

Features

- Content can change dynamically.
- Can use **server-side programming and databases**.
- Different users may see different content.
- Supports user interaction.
- Suitable for complex websites.
- Requires more processing than a simple static page.

Examples

- Online banking
- Student result portal
- Amazon shopping website
- Facebook/Instagram
- Online examination system
- Railway/flight booking system

Example:
When a student enters their **roll number**, the result portal retrieves the student's marks from a database and displays them.

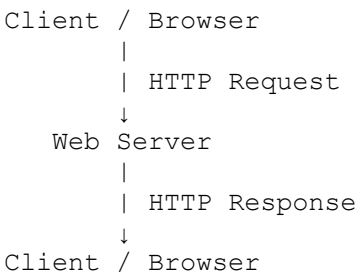
10. HTTP and HTTPS

HTTP

HTTP (HyperText Transfer Protocol) is an application-layer protocol used for communication between a **web browser (client)** and a **web server**.

It is mainly used to request and transfer web pages, images, videos, and other web resources.

Working of HTTP



Example

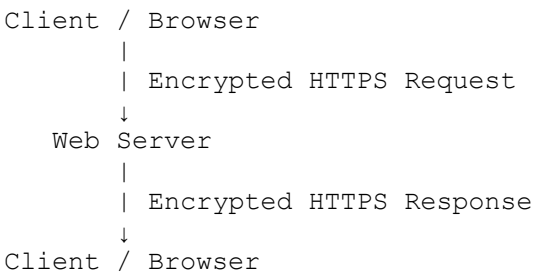
When you enter a website address in a browser, the browser sends an **HTTP request** to the server. The server sends the requested webpage back as an **HTTP response**.

http://example.com

HTTPS

HTTPS (HyperText Transfer Protocol Secure) is the **secure version of HTTP**. It protects communication between the browser and web server using **TLS (Transport Layer Security)**.

Working of HTTPS



The data exchanged between the browser and server is encrypted, making it difficult for unauthorized people to read or modify.

Example

`https://example.com`

Websites involving **passwords, banking, online shopping, and personal information** should use HTTPS.

Difference Between HTTP and HTTPS

HTTP	HTTPS
HyperText Transfer Protocol	HyperText Transfer Protocol Secure
Not encrypted	Encrypted
Less secure	More secure
Uses HTTP communication	Uses HTTP over TLS
Suitable for non-sensitive information	Suitable for sensitive information
URL starts with http://	URL starts with https://
Does not provide TLS security	Provides TLS security

11. Web Privacy

Web privacy means protecting a user's **personal information, online activities, and data** while using websites and Internet services.

It ensures that personal information is not collected, tracked, or shared without proper permission.

Web Privacy is Important

When we use websites, we may provide information such as:

- Name and email address
- Passwords
- Phone numbers
- Location
- Browsing history
- Online purchases
- Personal preferences

Protecting this information helps prevent misuse and unauthorized access.

Methods to Protect Web Privacy

1. **Use HTTPS**
   HTTPS encrypts communication between the browser and website.
2. **Use Strong Passwords**
   Use strong and unique passwords for different accounts.
3. **Enable Two-Factor Authentication (2FA)**
   Provides an additional layer of security.

4. **Manage Cookies**
   Block or delete unnecessary cookies and third-party tracking cookies.
5. **Avoid Suspicious Links**
   Do not click unknown links or provide personal information to suspicious websites.
6. **Check Privacy Settings**
   Review privacy settings in browsers, websites, and social media applications.
7. **Use Secure Networks**
   Avoid entering sensitive information on untrusted public Wi-Fi networks.

12. Internet Traffic

Internet traffic refers to the **flow and amount of data** transmitted through the Internet from one device or network to another at a particular time.

The data may include web pages, emails, videos, audio, files, messages, and other online information.

Problems Caused by High Internet Traffic

1. Network Congestion

Too much data can overload network links and routers.

2. Delay

Packets may take longer to reach their destination.

3. Packet Loss

Some packets may be dropped when network resources are overloaded.

4. Reduced Speed

Users may experience slower Internet speeds.

5. Poor Quality of Service

Video streaming, online gaming, and video calls may become unstable.

Methods to Manage Internet Traffic

- **Caching** – Frequently requested content is stored closer to users.
- **Web Proxies** – A proxy can cache and forward web content.
- **Server Farms** – Multiple servers share the workload.
- **CDNs** – Content is delivered from servers geographically closer to users.
- **Load Balancing** – Requests are distributed among multiple servers.
- **Traffic Management** – Network resources are controlled to reduce congestion.

13. Server Farms

server farm is a collection of multiple servers connected together and working as a group to handle a large number of requests from users.

Instead of depending on one server, the workload is distributed among several servers.

Example

When thousands of users access an online shopping website at the same time, one server may not be able to handle all requests. A **server farm** uses multiple servers to handle these requests.

Working of Server Farm

1. **User sends a request** to a website.
2. The request reaches the **load balancer**.
3. The load balancer checks the available servers.
4. It distributes the request to a suitable server.
5. The selected server processes the request.
6. The response is sent back to the user.

Advantages of Server Farms

1. High Performance

Multiple servers can process requests simultaneously.

2. Load Distribution

The workload is distributed among several servers, preventing one server from becoming overloaded.

3. High Availability

If one server fails, other servers can continue providing the service.

4. Scalability

More servers can be added when the number of users increases.

5. Reliability

The system continues to work even when some servers experience problems.

6. Faster Response

Requests can be processed by available servers, improving response time.

14. Web Proxies

A **Web Proxy** is an **intermediate server** that acts between a user and the web server. It receives the user's request, forwards it to the required web server, and sends the response back to the user.

Working of Web Proxy

1. The user requests a webpage through a browser.
2. The request first reaches the **web proxy**.
3. The proxy checks whether the requested content is already stored in its **cache**.
4. If available, the proxy sends the cached content directly to the user.
5. If not available, the proxy forwards the request to the web server.
6. The web server sends the response to the proxy.
7. The proxy may store the content in its cache.
8. The proxy sends the content to the user.

Functions of Web Proxy

1. Caching
Stores frequently requested webpages and files to provide them faster next time.

2. Reduces Bandwidth Usage
Cached content does not need to be downloaded repeatedly from the Internet.

3. Access Control
Organizations can control which websites users are allowed to access.

4. Filtering
Can block unwanted or restricted websites and content.

5. Security
Can provide an additional layer between users and external web servers.

6. Faster Access
Frequently requested content can be delivered from the proxy's cache.

15. Content Delivery Network (CDN)

A **Content Delivery Network (CDN)** is a network of **distributed servers located in different geographical locations** that delivers web content to users from a server that is close to them.

It helps websites deliver content **faster and more efficiently**.

A CDN can deliver:

- Web pages
- Images

- Videos
- Audio
- CSS and JavaScript files
- **Software and other downloadable files**

Working of CDN

1. A website stores its original content on an **origin server**.
2. Frequently requested content is copied to **CDN edge servers**.
3. A user requests content from a website.
4. The CDN identifies a nearby or suitable edge server.
5. If the content is available in the cache, the edge server sends it directly to the user.
6. If the content is not available, the CDN obtains it from the origin server.
7. The content may then be cached at the edge server for future requests.

Example

Suppose the main server of a video website is located in **the USA**, but a user is in **India**.

Without CDN:

```
India User → USA Server → India User
```

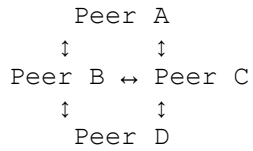
With CDN:

```
India User → Nearby CDN Server → India User
```

The CDN can deliver cached content from a location closer to the user, reducing delay.

16. Peer-to-Peer (P2P) Network

A **Peer-to-Peer (P2P) network** is a network in which computers (called **peers**) communicate and share resources directly with each other without depending completely on a central server.



Advantages of P2P Network

1. Easy Resource Sharing – Files, printers, and other resources can be shared directly.
2. No Central Server Required – The network does not completely depend on one central server.
3. Cost Effective – No need for expensive dedicated servers.
4. Scalable – New peers can be added easily.
5. Distributed System – Work and resources are distributed among multiple peers.

Applications of P2P Networks

1. **File Sharing** – Sharing large files between users.
2. **Video and Audio Sharing** – Distribution of multimedia content.
3. **Online Gaming** – Peers can communicate directly for multiplayer gaming.
4. **Communication** – Used in some voice and video communication systems.
5. **Content Distribution** – Helps distribute data among many users.

17. Evolution of Internet

The **Evolution of the Internet** refers to the gradual development of the Internet from a small research network into a **global network** used for communication, information sharing, business, entertainment, cloud services, and smart devices.

Stages of Internet Evolution

1. ARPANET – 1960s

- **ARPANET** was one of the earliest computer networks.
- It was developed for research and communication between institutions.
- It introduced the idea of connecting computers over a network.

2. Development of TCP/IP – 1970s–1980s

- **TCP/IP** protocols were developed to allow different networks to communicate with each other.
- TCP/IP became the basic communication foundation of the Internet.
- This helped create a network of interconnected networks.

3. World Wide Web – 1989 onwards

- The **World Wide Web (WWW)** made Internet information easier to access.
- It introduced web pages, hyperlinks, web browsers, URLs, and web servers.
- Users could easily access information through websites.

4. Mobile Internet

- The growth of smartphones and wireless networks allowed people to access the Internet anywhere.
- Mobile apps, mobile websites, messaging, and video services became common.

5. Cloud Computing

- Applications, storage, and computing resources started being provided through the Internet.
- Users can access services without storing everything locally.
- Examples: cloud storage, online applications, streaming services.

6. IoT and Modern Internet

- **Internet of Things (IoT)** connects devices such as smart watches, sensors, vehicles, and home appliances to the Internet.
- Modern networks support technologies such as **AI, 5G/6G, edge computing, and smart systems**.